



Temple

Informe técnico — Registro de entrenamientos de fuerza
PWA local-first y accesible (WCAG 2.2 AA)

Autor: Denis Lucian · **Fecha:** 14 de junio de 2026 · **Versión:** 1.3
Proyecto personal de portfolio · Coste total: 0 € (herramientas 100% gratuitas)

Temple — Informe Técnico

Registro de entrenamientos de fuerza · PWA local-first y accesible

Autor	Denis Lucian (denislucianwork@protonmail.com)
Fecha	14 de junio de 2026
Versión	1.3 (red social con cuentas, privacidad por publicación y RGPD en modo local; diseño de seguridad de producción en sección 17 y docs/SECURITY.md)
Repositorio	forjafit (proyecto personal de portfolio)
Coste total del proyecto	0 € (todas las herramientas son gratuitas)

1. Resumen ejecutivo

Temple es una aplicación web progresiva (PWA) de **registro de entrenamientos de fuerza**: el usuario apunta sus series, repeticiones y pesos en el gimnasio, consulta su historial, sus récords personales y la evolución de su progreso en gráficas. Tres principios la diferencian de las apps comerciales del mercado:

- Local-first / offline-first.** Los datos viven en el dispositivo del usuario (IndexedDB). La app funciona al 100% sin conexión — exactamente donde se usa: un gimnasio con mala cobertura. No hay cuentas, no hay servidores, no hay suscripciones.
- Propiedad de los datos.** Exportación e importación libre en JSON y CSV con un clic, sin paywall. Lo que las apps líderes cobran (rutinas ilimitadas, historial completo, export), Temple lo da gratis.
- Accesibilidad WCAG 2.2 AA.** Ningún competidor del nicho —comercial ni open source— compite en accesibilidad. Temple la trata como requisito de primera clase, auditada y documentada.

El proyecto se construye **por capas** (datos → dominio → interfaz → PWA/calidad), cada una con sus commits, sus tests y su documentación, para que el proceso sea tan presentable como el resultado. Este informe recoge la investigación de mercado que justifica cada decisión, la arquitectura, el plan de desarrollo y la estrategia para presentarlo a empresas.

2. Contexto y objetivos

2.1 Objetivos del proyecto

Objetivo	Cómo se mide
Aprender desarrollo web profesional moderno	Stack estándar de industria (React + TypeScript), arquitectura por capas explicable en entrevista
Funcionalidad real, no demo de juguete	App usable a diario en el gimnasio, 100% offline
Accesibilidad	WCAG 2.2 AA, Lighthouse Accessibility ≥ 95 , auditoría axe sin errores
Profesionalidad	Tests con Vitest, CI con GitHub Actions, commits convencionales, README tipo caso de estudio
Valor para el CV	Demo desplegada en un clic + repo público + métricas medibles
Coste cero	Solo herramientas y servicios gratuitos, sin tarjeta de crédito

2.2 Restricciones

- **Equipo de 1 persona** (desarrollo en solitario).
- **Presupuesto 0 €**: ninguna dependencia de APIs de pago, API keys con límites inviables o servicios que puedan apagarse.
- **La demo nunca puede estar caída**: si un reclutador la abre meses después, debe funcionar (esto descarta backends gratuitos que se pausan por inactividad).
- **Alcance controlado**: el mayor riesgo de un MVP no es técnico, es el *scope creep*. El alcance se fija con MoSCoW (sección 5.2) y lo que queda fuera se documenta como decisión deliberada.

3. Investigación de mercado

Metodología: investigación multi-agente en 6 dimensiones (mercado, APIs de datos, stack gratuito, valor para CV, accesibilidad, competidores) con **verificación adversarial** de las afirmaciones críticas contra documentación oficial (junio 2026). Las fuentes completas están en la sección 14.

3.1 El mercado y la oportunidad

- El mercado de apps de fitness ronda los **12.900 M\$ (2025)** con crecimiento anual del $\sim 13,5\%$; el segmento dominante es precisamente *workout & exercise*. El dominio es reconocible al instante por cualquier entrevistador.
- Las apps comerciales líderes **han endurecido sus paywalls**:
- **Strong** (gratis): máximo 3 rutinas y 3 ejercicios personalizados; sin export de datos. Pro: 4,99 \$/mes · 99,99 \$ lifetime. Pasó de compra única ~ 10 \$ a suscripción; sus reviews lo describen como

"shameful display of greed".

- **Hevy** (gratis): ~4 rutinas, ~3 meses de historial, 7 ejercicios personalizados. Premium: 23,99 \$/año. Requiere conexión a internet para varias funciones.
- **Jefit Elite**: 69,99 \$/año; usuarios reportan pop-ups que interrumpen el registro y navegación lenta.
- **MyFitnessPal** retiró el escáner de códigos de barras del plan gratis; Premium ~19,99 \$/mes.
- Las **quejas de usuarios** se concentran en 4 temas: suscripciones abusivas, datos secuestrados (export bloqueado), dependencia de internet en el gimnasio y UIs que se degradan.
- La tendencia técnica **local-first** (datos en el dispositivo, privacidad, sin dependencia de la nube) ha crecido con fuerza en 2023-2026, y los datos de salud son su caso de uso ideal por su sensibilidad.

3.2 Alternativas de proyecto evaluadas

Opción	Veredicto	Motivo
Tracker de fuerza (PWA local-first)	✓ Elegida	Hueco documentado (paywalls, offline, export), sin dependencias externas, dominio de lógica testeable (1RM, volumen, PRs), v1 sin backend = coste 0 y demo siempre viva
App de nutrición	✗ Descartada	El foso de MyFitnessPal es su base de datos de alimentos, no el código; replicar eso dispara el alcance. Alternativas gratuitas ya existen (Cronometer, FatSecret)
App de running sobre API de Strava	✗ Descartada	Strava restringió su API en nov-2024 (datos solo para el usuario autenticado, prohibido entrenar IA) y en 2026 introdujo niveles de pago. Riesgo de que el proyecto se rompa de un día para otro
Tracker de hábitos genérico	✗ Descartada	Sin diferenciación: es el "to-do list" de 2026, los reclutadores ven cientos

3.3 Competidores directos y hueco

Competidor	Tipo	Lección
wger (~6.200 ★, Django, AGPL)	Open source autoalojado	Valida el nicho, pero exige montar un servidor (1-3 h de setup Docker) y su UI resulta abrumadora. Su modelo de datos (rutina → día → ejercicio → serie) es buena referencia
LiftLog (~459 ★, React Native + TS)	Open source móvil	El modelo a imitar: local-first, sin login, rápido. Sus usuarios en Hacker News valoran exactamente eso: velocidad, datos locales, sin suscripción
Flexify (~393 ★, Flutter, MIT)	Open source móvil	"Lightning-quick": la velocidad de registro como propuesta de valor. Su predecesor (Massive) fue archivado: menos features = más supervivencia
FitNotes	Gratuita Android	Querida por su simplicidad y export CSV, pero UI anticuada y solo Android

El hueco: no existe una **PWA web moderna, accesible y local-first** de registro de fuerza. Los open source son apps móviles nativas o requieren servidor; los comerciales cobran por lo básico. Y **ninguno publicita accesibilidad** (cumplimiento WCAG, lector de pantalla, teclado) — hay casos documentados de apps de fitness inutilizables con lector de pantalla por miles de imágenes sin texto alternativo.

3.4 Propuesta de valor (la frase para el CV)

"Temple regala exactamente lo que Strong y Hevy cobran (rutinas ilimitadas, historial completo, export de datos), resuelve las 4 quejas principales de los usuarios (suscripciones, datos secuestrados, dependencia de internet, UIs lentas) y es el único tracker del nicho que compite en accesibilidad."

4. Estrategia de datos de ejercicios (decisión con historia legal)

Una app de gimnasio necesita un catálogo de ejercicios. La investigación evaluó las fuentes disponibles y la **verificación adversarial cambió la decisión inicial** — un ejemplo real de por qué se verifica:

Fuente	Estado verificado (jun-2026)	Veredicto
Free Exercise DB (yuhonas, "Unlicense", 800+ ejercicios)	 Refutado como "100% seguro" : el propio mantenedor admite por escrito (issue #2) que <i>"no tiene ni idea de dónde vienen las imágenes"</i> ; la evidencia upstream (wrkout/exercises.json #305) apunta a que fueron scrapeadas de bodybuilding.com. La licencia Unlicense del repo no puede convertir en dominio público imágenes de terceros	 No vendorizar imágenes; el JSON estructurado es riesgo bajo pero las instrucciones podrían arrastrar copyright
ExerciseDB / AscendAPI (11.000+ ejercicios con GIFs)	Producto comercial; su política prohíbe cachear y almacenar datos/medios, las URLs de medios rotan semanalmente, y el fork open source que liberó los datos recibió un DMCA el 23-abr-2026 (HTTP 451 verificado)	 Incompatible con offline-first y repo público
API Ninjas / MuscleWiki	API key obligatoria, tiers gratuitos residuales (5 resultados/petición; 500 calls/mes solo playground), datos propietarios no redistribuibles	 Descartadas
wger API (wger.de/api/v2)	 Gratuita, sin API key para lectura, datos CC-BY-SA 4.0, throttling documentado (120-300 req/min). Verificado en vivo: 851 ejercicios, 264 con traducción al español (~31%)	 Válida como referencia y enriquecimiento, con atribución
Catálogo propio	Los <i>nombres</i> de ejercicios son hechos no protegibles; las <i>instrucciones</i> se redactan desde cero en español	 Elegida

Decisión: Temple incluye un **catálogo propio de ~50 ejercicios fundamentales, escrito en español desde cero** (nombres, grupo muscular, material, instrucciones breves), almacenado como JSON en el repositorio y ampliable por el usuario con ejercicios personalizados **sin límite** (lo que Strong limita a 3 y Hevy a 7). Sin imágenes de terceros: iconografía propia por grupo muscular (SVG). Cero riesgo legal, cero API keys, cero dependencias de red, y nombres nativos en español — algo que **ninguna** fuente gratuita ofrece completo.

5. El producto: Temple

5.1 Usuarios y escenario

Persona que entrena fuerza en gimnasio o en casa y quiere registrar su progreso sin pagar suscripción, sin crear cuenta y sin depender de cobertura móvil. Usa el móvil entre serie y serie (pantalla táctil, una mano, prisa) o el ordenador para revisar su progreso.

5.2 Alcance MoSCoW

Must have (v1.0 — este proyecto):

#	Funcionalidad	Justificación (investigación)
F1	Registrar entrenamiento: ejercicio + series (reps × peso), mostrando lo que hiciste la última sesión y con botón "repetir serie anterior"	Feature núcleo de Strong/Hevy; además cumple WCAG 3.3.7 <i>Redundant Entry</i> — el único caso donde un criterio de accesibilidad es la feature estrella
F2	Rutinas (plantillas) ilimitadas	Strong limita a 3, Hevy a 4 — hueco directo
F3	Historial completo de sesiones, sin límite temporal	Hevy gratis recorta a ~3 meses
F4	Récords personales automáticos y 1RM estimado (fórmulas Epley y Brzycki)	Lógica de dominio pura, ideal para tests unitarios; analítica que los comerciales cobran
F5	Temporizador de descanso accesible (anuncios aria-live , no solo visual)	Feature core del nicho + diferenciador a11y
F6	Gráficas de progreso (volumen semanal, evolución de 1RM por ejercicio) con resumen textual + tabla de datos alternativa	Analítica premium en los comerciales; patrón de accesibilidad de gráficas (GOV.UK/Deque)
F7	Export/import JSON y export CSV en un clic, sin paywall	Queja nº 2 de usuarios: datos secuestrados
F8	PWA instalable y 100% offline (service worker + IndexedDB)	Queja nº 3: dependencia de internet en el gimnasio
F9	Tema claro/oscuro con contraste AA en ambos	WCAG 1.4.3/1.4.11; dark mode con gris #121212, no negro puro
F10	Ejercicios personalizados ilimitados, en español	Catálogo propio (sección 4)
F11	Generador de planes según objetivo (fuerza/hipertrofia/definición × días/semana × material × nivel)	Algoritmo propio basado en patrones de movimiento, determinista y testeado; las apps comerciales lo venden como IA premium

Fase 2 (implementada en las capas 6-8 — estado detallado en la sección 14, Roadmap):

- ✓ Perfil local y objetivos de calorías/macros (Mifflin-St Jeor).
- ✓ Nutrición: diario por comidas, Open Food Facts (nombre + código de barras), generador de dietas y escáner de macros por foto (Gemini, clave del usuario).

- ☒ Comunidad como **red social con cuentas**: registro/login/sesión, privacidad por publicación (pública/seguidores/privada), grafo de seguidores, gestión de cuenta y RGPD (export + borrado al olvido). En modo local de demostración, tras las interfaces `AuthService` y `SocialRepository` listas para Supabase, **documentando con honestidad que la seguridad real es del servidor** (ver sección 17 y `docs/SECURITY.md`).
- Pendiente de cuenta externa: nube real (Supabase Auth + Postgres con Row Level Security: identidad y privacidad impuestas en el servidor). Imágenes de ejercicios: cuando haya fuente con licencia limpia.

La regla sigue siendo anti-*scope creep*: **cada fase se construye solo cuando la anterior está terminada, testeada y verificada**, y el núcleo debe seguir funcionando 100% offline aunque las capas de red fallen.

5.3 Modelo de datos

Exercise	Routine	Session
└ id	└ id	└ id
└ name (es)	└ name	└ date
└ muscleGroup	└ exerciseIds[]	└ routineId?
└ equipment	└ notes	└ entries[]
└ instructions		└ exerciseId
└ isCustom		└ sets[]
└ createdAt		└ reps
		└ weightKg
		└ done
		└ notes

Derivados calculados (nunca almacenados): volumen por sesión/semana ($\sum \text{reps} \times \text{peso}$), 1RM estimado (Epley: $\text{peso} \times (1 + \text{reps}/30)$; Brzycki: $\text{peso} \times 36/(37 - \text{reps})$), récords personales por ejercicio (mejor serie por peso y por 1RM estimado).

6. Decisiones técnicas (stack)

Cada elección incluye la alternativa descartada y el porqué — el formato que piden las entrevistas.

Capa	Elección	Alternativas descartadas y motivo
Lenguaje	TypeScript (estricto)	JavaScript puro: entre el 72% y el 82% de las ofertas frontend exigen o prefieren TS (State of JS 2025: solo el 6% usa JS puro); usar TS es la decisión de mayor impacto en CV
Framework UI	React 19	Vue (~30% de demanda vs ~80% de React en España); Angular domina en banca española pero React gana en startups y remoto internacional — los conceptos transfieren
Build	Vite 8 (create-vite , template react-ts)	CRA está abandonado; Next.js añade servidor innecesario para una app 100% cliente
Persistencia	IndexedDB vía Dexie.js (patrón repositorio)	localStorage : límite ~5 MiB y API síncrona/bloqueante; IndexedDB da cientos de MB-GB. Backend (Supabase/Firebase): innecesario en v1, y Supabase free se pausa tras 7 días de inactividad — una demo caída ante un reclutador. El patrón repositorio permite cambiar IndexedDB por un backend sin tocar la UI: ese es el argumento de arquitectura para entrevistas
Gráficas	Recharts v3	Chart.js pesa menos (~67 KB vs ~136 KB) pero renderiza en <canvas> , invisible para lectores de pantalla; Recharts renderiza SVG semántico y su v3 trae accessibilityLayer (navegación por teclado). Prioridad del proyecto: accesibilidad
PWA	vite-plugin-pwa (Workbox, estrategia prompt)	Service worker manual: reinventar Workbox sin necesidad
Testing	Vitest 4 + React Testing Library + axe-core	Jest: Vitest comparte la config de Vite, es más rápido y es el estándar 2026 en proyectos Vite. Playwright E2E queda como capa futura documentada
Calidad	ESLint + eslint-plugin-jsx-a11y + Prettier	Caza errores de accesibilidad en el editor, antes del commit
CI/CD	GitHub Actions (lint + test + build en cada push)	Gratis sin límite de minutos en repos públicos ; el badge verde en el README es señal directa para reclutadores
Hosting	GitHub Pages (deploy automático desde Actions); Cloudflare Pages como alternativa documentada	Cloudflare Pages free es objetivamente el más generoso (ancho de banda ilimitado, 500 builds/mes, verificado); GitHub Pages (100 GB/mes soft) gana por simplicidad: todo en un ecosistema y el pipeline visible. Vercel Hobby descartado : sus fair use guidelines lo restringen a uso <i>no comercial</i> y pausa funciones hasta 30 días al exceder límites. Netlify free descartado : desde sep-2025 funciona por créditos (300/mes ≈ ~15 GB) y al agotarlos pausa todos los sitios de la cuenta
Datos de ejercicios	Catálogo propio en español (JSON vendorizado)	Ver sección 4: la verificación legal descartó las fuentes "gratuitas" habituales

Coste total: 0 €. Ningún servicio requiere tarjeta de crédito.

7. Arquitectura por capas

El requisito de "desarrollo por capas" no es solo orden de trabajo: es la arquitectura del código. Las dependencias apuntan **siempre hacia abajo** y cada capa se puede explicar, testear y sustituir de forma independiente.

CAPA 4 · PWA / Plataforma
service worker, manifest, instalación, offline
CAPA 3 · Interfaz (React)
vistas, componentes accesibles, rutas, gestión de foco, temas claro/oscuro
CAPA 2 · Dominio (TypeScript puro, sin React)
cálculos: 1RM, volumen, récords, estadísticas → funciones puras, 100% testeadas
CAPA 1 · Datos
modelos + repositorios (Dexie/IndexedDB), catálogo de ejercicios, export/import

Estructura de carpetas correspondiente:

```
src/
├─ data/           # CAPA 1: modelos, db (Dexie), repositorios, catálogo, export/import
├─ domain/         # CAPA 2: oneRepMax.ts, volume.ts, records.ts, stats.ts (+ tests)
├─ ui/             # CAPA 3: components/, views/, hooks/, theme
└─ pwa/            # CAPA 4: configuración del service worker (vite-plugin-pwa)
```

Regla de oro (y respuesta de entrevista): `domain/` no importa nada de `ui/` ni de `data/`; `ui/` consume `domain/` y `data/` a través de interfaces. Si mañana los datos pasan de IndexedDB a Supabase, solo cambia la implementación del repositorio.

8. Accesibilidad: requisitos WCAG 2.2 AA aplicados

WCAG 2.2 es el estándar vigente (oct-2023) y el **European Accessibility Act** obliga desde el 28-jun-2025 a los servicios digitales en la UE (vía EN 301 549, cuya actualización a WCAG 2.2 llega en 2026). Apuntar a 2.2 AA es la jugada segura y un valor de mercado real: la demanda de perfiles con WCAG creció ~45% interanual.

Criterios aplicados al dominio concreto de Temple:

Criterio WCAG 2.2 Aplicación en Temple	
3.3.7 Redundant Entry (A, nuevo)	Precargar peso/ reps de la última sesión del mismo ejercicio; botón "igual que la serie anterior". Requisito = feature estrella
2.5.8 Target Size Minimum (AA, nuevo)	Todos los controles $\geq 24 \times 24$ px CSS (objetivo real: 44 px en móvil): steppers +/-, checks de "serie hecha", botones de borrar
2.5.7 Dragging Movements (AA, nuevo)	Reordenar ejercicios de una rutina con botones "subir/bajar" accesibles por teclado, no solo arrastre
2.4.11 Focus Not Obscured (AA, nuevo)	<code>scroll-margin-top</code> para que la cabecera sticky nunca tape el elemento con foco
3.2.6 Consistent Help (A, nuevo)	Enlace de ayuda en posición consistente en todas las vistas
1.3.1 / 4.1.2 Formularios	<code><label></code> visible asociado a cada campo; reps: <code>type="text" inputmode="numeric" pattern="[0-9]*"</code> ; peso: <code>inputmode="decimal"</code> (patrón GOV.UK — nunca <code>type="number"</code> , que incrementa con el scroll y confunde a lectores de pantalla); errores con <code>aria-describedby</code> + <code>aria-invalid</code> , nunca solo color
SPA: gestión de foco	Al cambiar de vista, foco al <code><h1></code> (<code>tabindex="-1"</code>) + live region "Navegado a X"; skip link; foco devuelto al disparador al cerrar diálogos
4.1.3 Status Messages	Temporizador de descanso con anuncios <code>aria-live="polite"</code> en hitos (30 s, 10 s, fin), no solo cuenta visual
1.4.3 / 1.4.11 Contraste	4.5:1 texto, 3:1 UI y gráficas, verificado en ambos temas; dark mode <code>#121212</code> (el negro puro causa <i>halation</i> en astigmatismo)
Gráficas (1.1.1, 1.4.1)	Tres capas: resumen textual del insight en el cuerpo ("Tu press banca subió de 60 a 72,5 kg, +20%"), toggle "Ver como tabla" con tabla HTML real, y diferenciación que no dependa solo del color

Auditoría continua (todo gratis): `eslint-plugin-jsx-a11y` en el editor → tests con axe-core en CI → pasada manual con axe DevTools + Lighthouse → prueba con NVDA (lector de pantalla gratuito para Windows) y navegación solo-teclado. Las herramientas automáticas solo detectan el 30-40% de los fallos: la prueba manual queda documentada.

9. Plan de desarrollo por capas

Cada capa termina con: código + tests verdes + commits convencionales + sección propia en el README.

Capa	Entregable	Contenido
0. Cimientos	Proyecto compilando y desplegable	Scaffold Vite + React + TS estricto, ESLint (+ jsx-a11y), Prettier, Vitest, estructura de carpetas por capas, CI de GitHub Actions, HTML semántico base
1. Datos	Repositorios testeados	Modelos TypeScript, esquema Dexie, repositorios (sesiones, rutinas, ejercicios), catálogo de ~50 ejercicios en español, export/import JSON + export CSV
2. Dominio	Lógica pura 100% testeadas	<code>oneRepMax</code> (Epley/Brzycki), <code>volume</code> (por sesión/semana), <code>records</code> (PRs por ejercicio), <code>stats</code> (resúmenes para gráficas y textos accesibles)
3. Interfaz	App usable completa	Design system propio (tokens CSS, tema claro/oscuro), vistas: Entrenar, Historial, Rutinas, Ejercicios, Progreso, Ajustes; formulario de registro con precarga de última sesión; temporizador de descanso; gráficas Recharts + tablas alternativas; gestión de foco SPA
4. PWA + Calidad	Producto auditado	vite-plugin-pwa (manifest, iconos, offline), auditoría axe/Lighthouse documentada, verificación funcional, revisión de código
5. Documentación	Entregables finales	README caso de estudio, este informe en PDF, guía de despliegue gratuito

10. Estrategia de testing y CI

- **Pirámide de testing:** base amplia de tests unitarios sobre `domain/` (cálculos — rápidos y deterministas) y `data/` (repositorios con fake-indexeddb); tests de componentes con Testing Library sobre los flujos críticos (registrar una serie, precargar última sesión); chequeo de accesibilidad automatizado con axe-core en los tests de componentes.
- **CI en GitHub Actions** (gratis e ilimitado en repos públicos): `lint` → `typecheck` → `test` → `build` en cada push. Badge en el README.
- **E2E con Playwright:** documentado como capa futura (decisión de alcance, no de desconocimiento).

Por qué importa: las propias ofertas junior españolas piden testing con Jest/Testing Library, y muy pocos portfolios junior lo incluyen — relación coste/beneficio excelente.

11. Riesgos y mitigaciones

Riesgo	Probabilidad	Mitigación
<i>Scope creep</i> (el asesino nº 1 de MVPs)	Alta	Alcance MoSCoW cerrado (sección 5.2); lo descartado se documenta como decisión
El navegador borra IndexedDB (eviction)	Baja	<code>navigator.storage.persist()</code> + export JSON manual como copia de seguridad del usuario
Demo caída ante un reclutador	—	Eliminado por diseño: app estática sin backend, GitHub Pages no se pausa
Riesgo legal de datos de terceros	—	Eliminado por diseño: catálogo propio (sección 4)
Lock-in del esquema de datos propio	Baja	Export JSON con esquema versionado y documentado

12. Métricas de éxito (las que van al CV)

- Lighthouse: **Accessibility** ≥ 95 , Performance ≥ 90 , Best Practices ≥ 95 (capturas en el README).
- **0 violaciones** en axe DevTools en todas las vistas.
- Cobertura de tests de la capa de dominio: **100%** (es lógica pura; no hay excusa).
- Funciona **100% offline** tras la primera carga (verificable desconectando la red).
- Tamaño del bundle inicial documentado.
- Pipeline CI verde en cada commit de `main`.

13. Presentación a empresas

En el CV (1 página, formato "logré X usando Y con resultado Z"):

Temple — PWA de registro de entrenamientos, local-first y accesible · React, TypeScript, IndexedDB - Desarrollé una PWA offline-first de registro de fuerza con React 19 + TypeScript y arquitectura por capas (datos/dominio/UI), con la capa de dominio testeada al 100% con Vitest. - Implementé accesibilidad WCAG 2.2 AA (gestión de foco SPA, formularios con patrón GOV.UK, gráficas con alternativa textual): Lighthouse Accessibility ≥ 95 y 0 violaciones axe. - Monté CI/CD gratuito con GitHub Actions (lint, tests, build, deploy a GitHub Pages) con commits convencionales.

En LinkedIn: entrada en la sección Proyectos con título específico ("PWA de seguimiento de entrenamientos — offline-first, WCAG 2.2 AA"), enlace **primero a la demo**, después al repo.

En la entrevista, la narrativa en 90 segundos: problema (las apps de gimnasio cobran por lo básico y no funcionan sin cobertura) → investigación (verifiqué licencias de datasets y descarté los habituales por

riesgo de copyright) → arquitectura (por capas, patrón repositorio: puedo cambiar IndexedDB por un backend sin tocar la UI) → calidad (tests, CI, auditoría de accesibilidad documentada) → resultado (demo instalable que funciona en modo avión).

14. Roadmap de producto: visión ampliada

La visión a largo plazo es una **plataforma fitness completa y diferenciada**: lo que el mercado ya ofrece (nutrición, social, generación inteligente) pero sin paywalls sobre lo básico, con accesibilidad real y con el núcleo siempre funcional sin conexión. Cada fase usa exclusivamente herramientas con tier gratuito verificado y se apoya en la arquitectura por capas: **lo local sigue siendo la fuente de verdad; la red es una capa opcional que enriquece.**

Fase	Qué añade	Herramienta gratuita (verificada)	Estado y notas
v1.1 ✓	Generador de planes según objetivo (fuerza/hipertrofia/definición, 2-5 días, material, nivel)	Algoritmo propio en <code>src/domain</code> (puro, testeado)	Hecha. Sin dependencias
v1.2 ✓	Perfil local + objetivos de calorías y macros	Mifflin-St Jeor + factores de actividad + reparto de macros, en <code>src/domain/nutritionTargets.ts</code> (testeado con valores de referencia)	Hecha. El perfil vive en el dispositivo; será la semilla del perfil en la nube
v1.3 ✓	Diario de calorías/macros, búsqueda de alimentos, generador de dietas y escáner por foto	Open Food Facts (sin key; nombre + código de barras, con caché local y mensajes que distinguen sin-conexión del rate limit) · dietas resolviendo gramos con sistema lineal 3×3 (−1,2% verificado) · Gemini free tier con clave del propio usuario guardada solo en su dispositivo	Hecha. Catálogo propio de ~70 alimentos en español como base offline
v1.4 ✓	Comunidad: feed, publicar rutina/sesión, me gusta y comentarios	Modo local sobre IndexedDB con publicaciones de ejemplo; interfaz <code>SocialRepository</code> como contrato	Hecha en modo demostración: experiencia completa sin servidor
v1.5 ✓	Herramientas premium del mercado: calculadora de discos y calentamiento, progresión sugerida, duración de sesión, medidas corporales con sync del perfil, rachas + heatmap, 12 logros derivados, hidratación y copiar-día	Todo dominio puro propio (<code>gymTools</code> , <code>consistency</code>) + DB v3; los logros se derivan de los datos reales (nada que desincronizar)	Hecha. Es exactamente lo que Hevy/Strong venden en premium, gratis y offline
v1.6 ✓	Lo que destaca en Hevy y Cal AI (2026), investigado con fuentes: tipos de serie (calentamiento fuera del volumen/PR), RPE, reordenar y notas; en nutrición, descripción por texto/voz, foto de etiqueta, Nutri-Score (algoritmo 2023) y objetivo de peso con fecha	Dominio puro testeado (<code>nutriScore</code> , <code>weightGoal</code> , <code>isWorkingSet</code>); IA reutilizando Gemini; Web Speech API para la voz	Hecha. Reimplementa local-first las fórmulas que Cal AI/MacroFactor esconden, con el estándar público equivalente y auditable
v1.7 ✓	Red social con cuentas: registro/login/sesión, privacidad por publicación (pública/seguidores/privada), grafo de seguidores, gestión de cuenta y RGPD (export + borrado al olvido)	Modo local sobre IndexedDB (DB v4) tras las interfaces <code>AuthService</code> y <code>SocialRepository</code> ; hash PBKDF2-SHA256 (Web Crypto); filtro de feed como función de dominio pura (<code>visiblePosts</code>)	Hecha en modo demostración, documentando con honestidad que no es seguridad real (el hash y el filtro viven en el cliente). Diseño de producción en SECURITY.md

Fase	Qué añade	Herramienta gratuita (verificada)	Estado y notas
v2.0	Autenticación real + comunidad compartida con privacidad impuesta por el servidor	Supabase Free: Postgres 500 MB, Auth 50.000 MAU, Realtime. Supabase Auth (email+contraseña con bcrypt en servidor, Google OAuth → cumple WCAG 3.3.8) y Row Level Security en Postgres replicando <code>visiblePosts</code> por fila	Pendiente (requiere crear la cuenta). El free tier se pausa tras 7 días de inactividad → cron semanal de keep-alive en GitHub Actions. RLS en todas las tablas desde el día 1; la <code>service_role</code> key nunca en el cliente. Gracias al patrón repositorio, solo se escribe el adaptador: la UI no cambia. Modelo de amenazas OWASP y checklist en SECURITY.md
v2.1	Proxy para el escáner IA	Supabase Edge Functions (500K invocaciones gratis/mes)	En la app personal actual la clave Gemini es del propio usuario y vive solo en su dispositivo (acceptable); en un despliegue multiusuario debe moverse a un proxy
v2.2	Imágenes/ilustraciones de ejecución de ejercicios	Las 357 imágenes de wger (CC-BY-SA 4.0, atribución por imagen — verificado: solo 83 son de Everkinetic) o ilustraciones SVG propias	Nunca datasets scrapeados: hay DMCA confirmado (abr-2026) contra redistribuidores de ExerciseDB

El orden seguido: primero todo lo que no requiere cuentas de terceros (v1.1-v1.4, hechas — la app completa funciona hoy sin ningún servicio externo obligatorio); después la base de identidad (auth) que convierte la comunidad local en compartida; y las imágenes cuando haya presupuesto de tiempo para hacerlas con licencia limpia.

Implicación arquitectónica clave (argumento de entrevista): gracias al patrón repositorio de la Capa 1, añadir Supabase no toca la UI ni el dominio — se añade una implementación `SupabaseRepository` junto a la actual `DexieRepository` con sincronización tipo *outbox* (lo local manda, la red reconcilia). El informe original ya anticipaba esta evolución.

15. Benchmarking competitivo: Hevy y Cal AI (2026)

Antes de la fase 4 se hizo una investigación con fuentes de las dos apps de referencia del mercado para identificar qué funciones destacan y cuáles aún no tenía Temple. El criterio de selección fue **impacto/esfuerzo y valor demostrable en una entrevista**, priorizando lo que se puede implementar como función pura testeable.

De Hevy (registro de fuerza):

Función	Qué aporta	Implementación en Temple
Tipos de serie (calentamiento/drop/fallo)	El calentamiento NO debe contar como volumen efectivo: contarlo da estadísticas mentirosas	<code>isWorkingSet()</code> excluye el calentamiento del volumen, los récords y las estadísticas. Campo opcional → retrocompatible
RPE / RIR	Autorregulación moderna (progresar por esfuerzo, no solo por peso)	Campo opcional 6-10 por serie
Reordenar / notas	Realismo de gimnasio (máquina ocupada, señales de técnica)	Botones 1/↓ accesibles (WCAG 2.5.7) y nota por ejercicio

De Cal AI / MacroFactor (nutrición con IA):

Función	Qué aporta	Implementación en Temple
Describir comida por texto/voz	Registro en segundos sin foto ni búsqueda	Mismo Gemini que la foto con input de texto; voz con Web Speech API (gratis)
Foto de etiqueta nutricional	Resuelve productos que no están en Open Food Facts	OCR estructurado con Gemini → valores por 100 g
Health score	Feedback cualitativo ("¿esto es sano?")	Nutri-Score oficial 2023 (público y auditable), no la fórmula opaca de Cal AI
Objetivo de peso con fecha	"¿Cuándo llego a mi meta?"	<code>projectWeightGoal</code> con la constante estándar de 7700 kcal/kg

Decisión transversal: Cal AI y MacroFactor no publican sus fórmulas de health score ni de proyección. En lugar de inventar una caja negra, Temple implementa los **estándares públicos equivalentes** (Nutri-Score 2023, 7700 kcal/kg) como funciones puras testeadas — más honesto, auditable y defendible. Fuentes en la sección 16.

16. Fuentes principales

Mercado y competidores: polarismarketresearch.com (fitness app market) · gymgod.app/blog/strong-vs-hevy · setgraph.app (Hevy vs Strong 2026) · github.com/wger-project/wger · github.com/LiamMorrow/LiftLog · github.com/brandonp2412/Flexify · news.ycombinator.com/item?id=38283760 · apps.apple.com (reviews Strong) · trustpilot.com/review/www.jefit.com

Datos y licencias: github.com/yuhonas/free-exercise-db (issues #2, #12) · github.com/wrkout/exercises.json (issue #305) · github.com/github/dmca (2026-04-23-exercisedb.md) · docs.ascendapi.com (caching/ratelimiting) · wger.de/api/v2 (verificación en vivo) · world.openfoodfacts.org/data · api-ninjas.com/pricing · musclewiki.com/api-terms

Benchmarking Heavy/Cal AI (fase 4): heavyapp.com/features (supersets, exercise-programming-options, sets-per-muscle-group) · help.heavyapp.com (smart superset scrolling) · calai.app · apps.apple.com (Cal AI) · snapcalorie.com · macrofactor.com/algorithm-accuracy (adaptive TDEE) · eclarion.com/nutriscore-calculator/methodology (Nutri-Score 2023) · eurofins.de (Nutri-Score update 2023) · myfitnesspal.com (target date) · [wellness sobre 7700 kcal/kg](#)

Stack y hosting: developers.cloudflare.com/pages/platform/limits · vercel.com/docs/plans/hobby · vercel.com/docs/limits/fair-use-guidelines · docs.netlify.com (credit-based plans) · docs.github.com (Pages limits, Actions billing) · supabase.com/pricing · firebase.google.com/pricing · vite.dev/releases · vitest.dev · github.com/vite-pwa/vite-plugin-pwa · pkgpulse.com (Recharts vs Chart.js)

CV y mercado laboral: stateofjs.com (2025) · infoq.com (State of JS survey) · tecnoempleo.com (informe junio 2026) · webportfolios.dev (junior portfolio guide) · getmanfred.com · dev.to (recruiters 2025-26) · a11yjobs.com

Accesibilidad: w3.org/WAI (new-in-22, Understanding 2.5.8/2.5.7/3.3.7) · technology.blog.gov.uk (input type number) · accessibility.blog.gov.uk (text descriptions for data visualisations) · deque.com (interactive charts, EAA) · gatsbyjs.com (accessible client routing) · commission.europa.eu (EAA) · github.com/recharts/recharts/wiki (accessibility)

Seguridad y privacidad: owasp.org/Top10 · supabase.com/docs/guides/auth · supabase.com/docs/guides/database/postgres/row-level-security · cheatsheetseries.owasp.org (Password Storage, Authentication) · gdpr.eu (arts. 5, 9, 15, 16, 17, 20, 25)

17. Seguridad y privacidad: del modo local a la nube

La capa de comunidad convierte Temple en una **red social de fitness con cuentas**. El reto de hacerlo en un proyecto local-first es la **honestidad**: una app sin servidor no puede ofrecer seguridad real entre usuarios, y fingir lo contrario sería un error de criterio delatante en una entrevista. La decisión de diseño es separar el **contrato** (estable) de la **implementación** (local hoy, Supabase mañana) y documentar dónde está exactamente la frontera cliente/servidor.

Lo que está implementado (modo local), con sus límites declarados:

- **Cuentas tras la interfaz** `AuthService` (`LocalAuthService`): registro, login, logout y sesión. Contraseña derivada con **PBKDF2-SHA256 + sal** (Web Crypto). Se documenta explícitamente que **un hash en cliente no es seguridad real** (las DevTools dan acceso total a IndexedDB): solo evita guardar la contraseña en claro. Incluye **anti-enumeración de usuarios** (mensaje genérico + hash de coste constante en la rama "el usuario no existe").
- **Privacidad por publicación** (pública / solo seguidores / privada) resuelta por `visiblePosts()`, **función de dominio pura y testeada**. En el cliente sirve para no pedir lo que no se podrá ver; en producción **se replica línea por línea en una política RLS** que es la que de verdad protege.
- **Grafo social** (seguir/dejar de seguir) y **gestión de cuenta en Ajustes**: editar perfil, perfil privado, cambiar contraseña y **borrar la cuenta (derecho al olvido, RGPD)** en una transacción.

Lo que diseña la fase nube (en `docs/SECURITY.md`):

- **Supabase Auth** (JWT firmado + bcrypt en servidor, email+contraseña y Google OAuth) como única autoridad de autenticación.
- **Postgres con Row Level Security**: cada tabla con datos de usuario lleva `enable row level security` y políticas *deny-by-default*; la lectura de publicaciones reproduce `visiblePosts` con `auth.uid()` evaluado por fila. La `service_role` key **nunca** toca el cliente.
- **Modelo de amenazas OWASP Top 10** (control de acceso roto, inyección, fallos de autenticación, mala configuración...) con mitigaciones concretas, y **cumplimiento del RGPD** (minimización, datos de salud que no salen del dispositivo, acceso/portabilidad/supresión/rectificación, privacidad desde el diseño).
- **Accesibilidad de la autenticación** (WCAG 2.2 — 3.3.8): `autocomplete`, se permite pegar la contraseña y sin CAPTCHA de puzzles como requisito de entrada.

Argumento de entrevista: el valor no está en aparentar seguridad, sino en **saber dónde está la frontera entre cliente y servidor y construir la app para cruzarla sin reescribir la UI**. El detalle completo — esquema SQL, políticas RLS, checklist de paso a producción— está en `docs/SECURITY.md`.

Informe generado como parte del proyecto Temple. La versión PDF de este documento se genera desde este Markdown (ver README, sección Documentación).